



HASHDIT

HashDit

Smart Contract Security

Venus FRV Vault Naming Security Audit Report

Venus

1 July 2026 – 2 July 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	1 July 2026 – 2 July 2026
Overall Risk [Optional]	Informational
Project Name	Venus
Github	https://github.com/VenusProtocol/fixed-rate-vaults/pull/10
Branch	feat/vpd-1488

Smart Contract List

No.	Contract Name	Verdict	Details
1	InstitutionalLoanVault.sol	Green	
2	InstitutionalVaultController.sol	Informational	[I01]-Ack [I02]-Fixed [I03]-Ack

Findings

[I01] getAggregatedVaultStates() Requires a Paired Override for the Legacy Vault – Recommend Adding to the Release Checklist

Contract InstitutionalVaultController.sol

Severity Informational

Description

_resolveInstitutionName calls the vault's institutionName() directly when no override is set; legacy vaults lack this function, causing the call to revert and taking down all of getAggregatedVaultStates() with it. This is a known, deliberately designed-for behavior: the interface comment on institutionName() explicitly states "Reverts on legacy vaults deployed without this field," a developer review comment notes "there's currently only a single vault deployed" as the rationale for explicit handling over a blanket try/catch, and the PR's own fork test is specifically built to assert "reverts before the override is set, succeeds after."

A live chain query confirms the sole vault currently registered on BSC mainnet lacks this function, and actually running that fork test reproduces the revert. BSC testnet has zero registered vaults, so this step cannot be verified through a normal testnet rehearsal.

JavaScript

```
function getAggregatedVaultStates() external view returns
(VaultStateInfo[] memory) {
    uint256 len = allVaults.length;
    VaultStateInfo[] memory infos = new VaultStateInfo[](len);
    for (uint256 i; i < len; ++i) {
        address v = allVaults[i];
        IIInstitutionalLoanVault vault = IIInstitutionalLoanVault(v);
        infos[i] = VaultStateInfo({
            vault: v,
            state: vault.state(),
            institutionOperator:
                vault.institutionalConfig().institutionOperator,
            totalRaised: vault.runtime().totalRaised,
            outstandingDebt: vault.outstandingDebt(),
            institutionName: _resolveInstitutionName(v) // one vault's
failure reverts the entire loop
        });
    }
    return infos;
}

// InstitutionalVaultController - _resolveInstitutionName
```

```
function _resolveInstitutionName(address vault) internal view returns
(string memory name) {
    name = institutionNameOverride[vault];
    if (bytes(name).length != 0) return name;
    return IInstitutionalLoanVault(vault).institutionName(); //
reverts on legacy vaults (known behavior)
}
```

Impact:

No fund-safety impact – affects only this one read-only aggregator's availability; no other on-chain contract in the repo depends on it. If the implementation upgrade and the override-setting are not executed in the same transaction, there is a window during which this function reverts for everyone until someone manually calls the already-built-and-tested `setInstitutionNameOverride`. The mechanism itself is already implemented and tested – this is a release-process reminder, not a code defect.

1. Make "upgrade the Controller implementation" and "call `setInstitutionNameOverride` for the legacy vault" a mandatory paired step in the same release checklist/VIP to avoid any gap window.
2. Since testnet currently has no registered vaults and cannot reproduce this scenario, make this PR's own mainnet fork test a mandatory pre-release check.
3. (Optional, more robust fix) Have `_resolveInstitutionName` isolate per-vault failures via a low-level staticcall with a success check, so a missing getter on any single vault no longer takes down the whole aggregator regardless of whether an override was set in time:

Recommendation

JavaScript

```
function _resolveInstitutionName(address vault) internal view returns
(string memory name) {
    name = institutionNameOverride[vault];
    if (bytes(name).length != 0) return name;
    (bool ok, bytes memory data) =
    vault.staticcall(abi.encodeWithSignature("institutionName()"));
    return ok ? abi.decode(data, (string)) : "";
}
```

Status

Acknowledged. This addresses the deployment-time window described in this finding – bundling the override into the same transaction as the upgrade eliminates the gap where `getAggregatedVaultStates()` would otherwise revert.

[I02] setInstitutionName and setInstitutionNameOverride Lack On-Chain Guardrails; a Mistaken Override Cannot Be Undone

Contract InstitutionalVaultController.sol

Severity Informational

_resolveInstitutionName uses the override whenever it's non-empty, and setInstitutionNameOverride — the only function that can modify it — rejects an empty string. So once set to any non-empty value, the override can only be replaced with another non-empty value, never cleared. There's also no on-chain check limiting setInstitutionNameOverride to genuinely legacy vaults; it succeeds on any registered vault, including new ones. Combined with the two functions' near-identical names and signatures, this creates a real risk of misuse: if a vault has an override set and later gets a real rename via setInstitutionName, _resolveInstitutionName still always returns the stale override — the rename never actually takes effect.

JavaScript

```
// InstitutionalVaultController - _resolveInstitutionName /
setInstitutionNameOverride / setInstitutionName
function _resolveInstitutionName(
    address vault
) internal view returns (string memory name) {
    name = institutionNameOverride[vault];
    if (bytes(name).length != 0) return name; // non-empty override
    always wins, with no way to opt back out
    return IIInstitutionalLoanVault(vault).institutionName();
}

function setInstitutionNameOverride(
    address vault,
    string calldata newName
) external {
    _checkAccessAllowed("setInstitutionNameOverride(address,string)");
    if (!isRegistered[vault]) revert VaultNotRegistered();
    if (bytes(newName).length == 0) revert InvalidConfig(); // cannot
    be cleared - irreversible
    string memory oldName = institutionNameOverride[vault];
    if (keccak256(bytes(newName)) == keccak256(bytes(oldName))) revert
    InstitutionNameUnchanged();
    emit InstitutionNameOverrideUpdated(vault, oldName, newName);
    institutionNameOverride[vault] = newName;
}

function setInstitutionName(
    address vault,
    string calldata newName
) external {
    _checkAccessAllowed("setInstitutionName(address,string)");
    if (!isRegistered[vault]) revert VaultNotRegistered();
    if (bytes(newName).length == 0) revert InvalidConfig();
    string memory oldName =
    IIInstitutionalLoanVault(vault).institutionName();
```

Description

```

        if (keccak256(bytes(newName)) == keccak256(bytes(oldName))) revert
InstitutionNameUnchanged();
        emit InstitutionNameUpdated(vault, oldName, newName);
        IIInstitutionalLoanVault(vault).setInstitutionName(newName); //
never clears institutionNameOverride[vault] - the rename has no effect
on the aggregated view
    }

```

Impact:

No fund-safety impact — a display-layer data-consistency issue. Requires a governance operational mistake to trigger, does not happen automatically, and is confined to trusted ACM-gated roles with no external attack surface. Once triggered, any rename performed via setInstitutionName never takes effect in getAggregatedVaultStates() — callers only ever see the stale override value, with no on-chain path to restore the resolver to reading the vault's own name.

Have setInstitutionName clear the vault's institutionNameOverride in the same call after a successful rename (Controller modifying its own storage — no cross-contract permission concern):

JavaScript

```

function setInstitutionName(
    address vault,
    string calldata newName
) external {
    _checkAccessAllowed("setInstitutionName(address,string)");
    if (!isRegistered[vault]) revert VaultNotRegistered();
    if (bytes(newName).length == 0) revert InvalidConfig();
    string memory oldName =
IIInstitutionalLoanVault(vault).institutionName();
    if (keccak256(bytes(newName)) == keccak256(bytes(oldName))) revert
InstitutionNameUnchanged();
    emit InstitutionNameUpdated(vault, oldName, newName);
    IIInstitutionalLoanVault(vault).setInstitutionName(newName);
    delete institutionNameOverride[vault]; // keep the two paths in
sync
}

```

Recommendation

Status

Fix verified in PR #11 (commit [cd804d0](#)). The empty-string restriction was removed, allowing the override to be cleared and _resolveInstitutionName to fall back to the vault's own name again — this resolves the irreversibility issue. Note: verified via mainnet fork test that clearing the override on a vault whose implementation lacks institutionName() (i.e. the known legacy vault) will cause getAggregatedVaultStates() to revert again. This is non-blocking and self-correcting (re-setting a non-empty override restores it immediately) — recommend a one-line deployment runbook note not to clear the override on vaults known to lack institutionName().

[I03] After the upgrade, createVault is bricked until governance re-grants its permission, because the permission is keyed to the function signature and the signature changed

Contract InstitutionalVaultController.sol

Severity Informational

Venus's access manager grants permission for an exact function-signature string; this PR added a 6th argument to createVault, so the string changed, and the old grant no longer matches — meaning no one can create vaults after the upgrade until governance grants the new signature.

Description

```
JavaScript
// InstitutionalVaultController.sol
function createVault(
    VaultConfig calldata _vaultConfig,
    InstitutionalConfig calldata _instConfig,
    RiskConfig calldata _riskConfig,
    string calldata _name,
    string calldata _symbol,
    string calldata _institutionName           // <-- new 6th argument
    added by this PR
) external returns (address vault) {
    // @audit the checked signature string changed to the 6-arg form
    // below. The ACM grant that was
    // issued for the OLD 5-arg string
    //
    "createVault(VaultConfig,InstitutionalConfig,RiskConfig,string,string)"
    // no longer matches, so EVERY createVault call reverts here
    // until governance grants the
    // new string. The old grant is silently orphaned (still
    // "exists" but no longer applies).

    _checkAccessAllowed("createVault(VaultConfig,InstitutionalConfig,RiskCon
fig,string,string,string)");
    _validateVaultConfig(_vaultConfig, _instConfig, _riskConfig);
    if (bytes(_institutionName).length == 0) revert InvalidConfig();
    ...
}
```

On-chain verification: Querying live BSC mainnet confirms this is not theoretical — the Venus governance timelock (0x939bD8...ab6396) currently holds the ACM role for the old 5-argument createVault signature on the live Controller, but holds no role at all for the new 6-argument signature this PR introduces. The one address able to create vaults today will lose that ability the moment the Controller is upgraded, until a separate permission grant is issued.

Worked example:

	Governance upgrades the controller proxy to this PR but forgets to touch the ACM. An operator calls createVault(...). The call reverts inside _checkAccessAllowed because no one holds permission for the new 6-arg signature string. Nobody can create vaults until governance runs giveCallPermission(..., "createVault(VaultConfig,InstitutionalConfig,RiskConfig,string,string,string)", caller). (The PR's fork test does exactly this in _grantControllerPermissions, confirming the team knows the grant is required.) Impact: If the new grant is missed at upgrade time, new-vault creation is bricked until it is added. Purely an operational/migration hazard, no funds at risk, but easy to overlook because the stale 5-arg permission still appears to exist.
Recommendation	Add to the upgrade runbook, in the same governance batch as the proxy upgrade: grant createVault(VaultConfig,InstitutionalConfig,RiskConfig,string,string,string) plus the two new permissions setInstitutionName(address,string) and setInstitutionNameOverride(address,string). Optionally revoke the now-dead 5-arg grant to keep the ACM state clean.
Status	Acknowledged. Venus Protocol will include the new ACM permissions and the createVault signature change in the same upgrade VIP.